Algorithms

COMP2711 Discrete Mathematical Tools

Mingxun Zhou (mingxunz@ust.hk)

Lecture roadmap

1. **Problems and algorithms**
   Problem specification; instances; algorithms and correctness.

2. **Algorithm analysis**
   Basic operations; exact counts; worst-case running time.

3. **Asymptotic analysis**
   Growth rates; upper and lower bounds; tight bounds.

1. **Problems and algorithms**

   Problem specification; instances; algorithms and correctness.

2. **Algorithm analysis**

   Basic operations; exact counts; worst-case running time.

3. **Asymptotic analysis**

   Growth rates; upper and lower bounds; tight bounds.

Problems and instances

Problem What should we compute?

Problems and instances

Problem What should we compute?

GCD problem

Input Nonnegative integers a, b, not both zero.

Output The greatest common divisor $\gcd(a, b)$.

Problems and instances

Problem What should we compute?

GCD problem

Input Nonnegative integers a, b, not both zero.

Output The greatest common divisor $\gcd(a, b)$.

Instance One specific input.

$(a, b) = (43, 17)$. The required output is 1.

Algorithms and correctness

Algorithm A step-by-step method for solving a problem.

Algorithms and correctness

Algorithm A step-by-step method for solving a problem.

It must **stop** with a **correct** answer for every valid input (the most basic definition).

Algorithms and correctness

Algorithm A step-by-step method for solving a problem.

It must **stop** with a **correct** answer for every valid input (the most basic definition).

Example: Euclidean algorithm

While $b \neq 0$, replace (a, b) by $(b, a \bmod b)$. Return a.

$$(43, 17) \longrightarrow (17, 9) \longrightarrow (9, 8) \longrightarrow (8, 1) \longrightarrow (1, 0).$$

The algorithm always returns $\gcd(a, b)$ for input a, b.

Pseudocode: Describing an algorithm

Pseudocode Write the steps clearly, without choosing a programming language.

Pseudocode: Describing an algorithm

Pseudocode Write the steps clearly, without choosing a programming language.

Input: Nonnegative integers a, b, not both zero. **Output:** $\gcd(a, b)$.

EuclideanGCD(a, b)

if $a < b$: swap(a, b)
while $b \neq 0$ **do**
    $(a, b) \leftarrow (b, a \bmod b)$
return a

Pseudocode: Describing an algorithm

Pseudocode Write the steps clearly, without choosing a programming language.

Input: Nonnegative integers a, b, not both zero. **Output:** $\gcd(a, b)$.

EuclideanGCD(a, b)

if $a < b$: swap(a, b)
while $b \neq 0$ **do**
    $(a, b) \leftarrow (b, a \bmod b)$
return a

Update both entries together, using the old values of a and b.

Pseudocode and executable code

The same Euclidean algorithm, written in two ways.

Pseudocode

procedure EuclideanGCD(a, b)
if $a < b$: swap(a, b)
while $b \neq 0$ **do**
    $(a, b) \leftarrow (b, a \bmod b)$
return a

Written for people; notation can vary.

Pseudocode and executable code

The same Euclidean algorithm, written in two ways.

Pseudocode

procedure EuclideanGCD(a, b)
if $a < b$: swap(a, b)
while $b \neq 0$ **do**
    $(a, b) \leftarrow (b, a \bmod b)$
return a

Written for people; notation can vary.

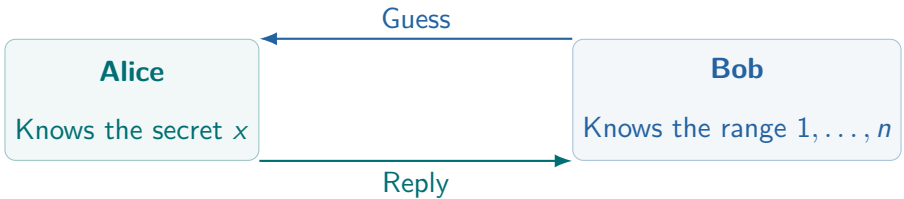
Python

```
def gcd(a, b):
    if a < b:  a, b = b, a
    while b != 0:
        a, b = b, a % b
    return a
```

Follows Python's syntax; runs on a computer.

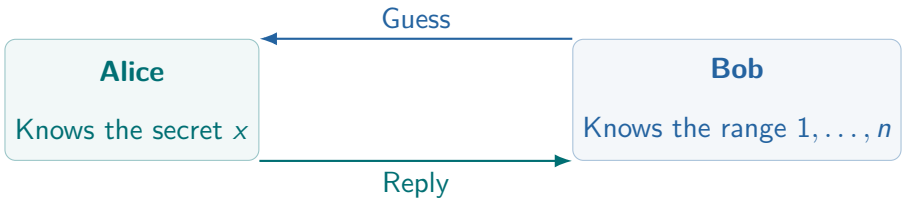
Case study: Finding the secret number

Problem Find Alice's secret integer $x \in \{1, \ldots, n\}$.



Case study: Finding the secret number

Problem Find Alice's secret integer $x \in \{1, \ldots, n\}$.

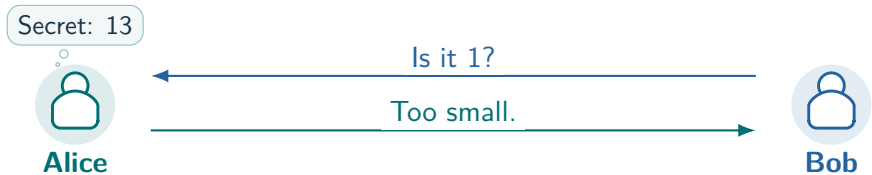


Alice replies **"too small"**, **"too big"**, or **"correct"**.

Instance $n = 16$, secret $x = 13$ (unknown to Bob).

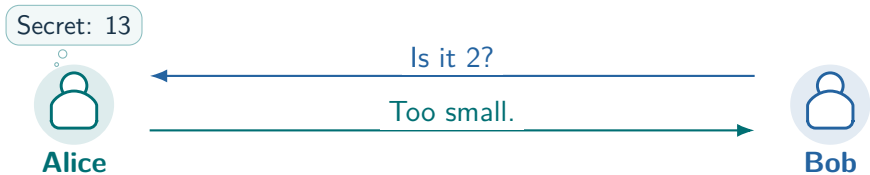
Algorithm 1: Linear search

$n = 16$. Bob guesses $1, 2, 3, \ldots$ until Alice says "correct".



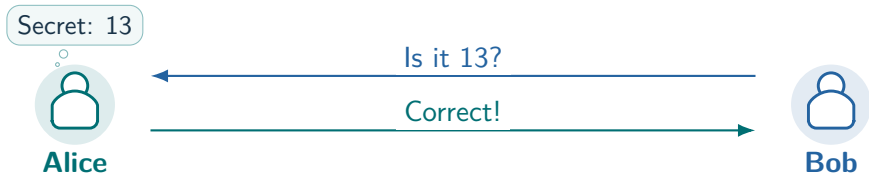
Algorithm 1: Linear search

$n = 16$. Bob guesses $1, 2, 3, \ldots$ until Alice says "correct".



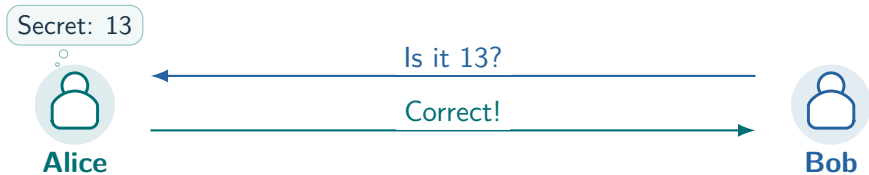
Algorithm 1: Linear search

$n = 16$. Bob guesses $1, 2, 3, \ldots$ until Alice says "correct".



Algorithm 1: Linear search

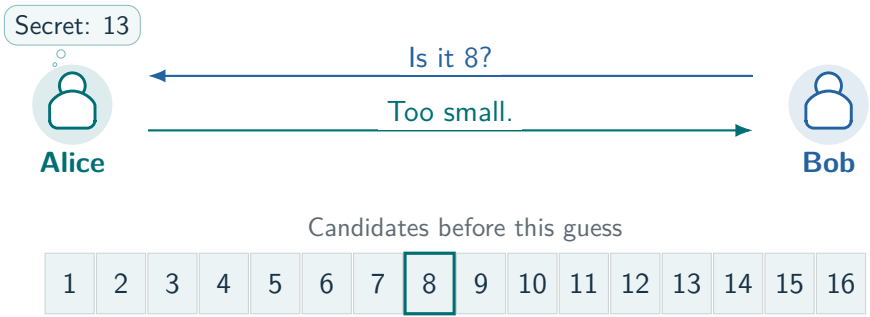
$n = 16$. Bob guesses $1, 2, 3, \ldots$ until Alice says "correct".



By guessing in order, Bob rules out one new number at a time.

Algorithm 2: Binary search

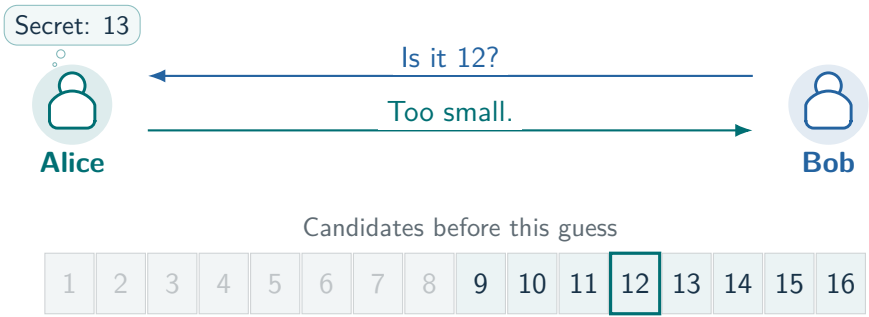
Guess the middle; keep the half indicated by Alice's reply.



Keep $9, \ldots, 16$: every smaller number is ruled out.

Algorithm 2: Binary search

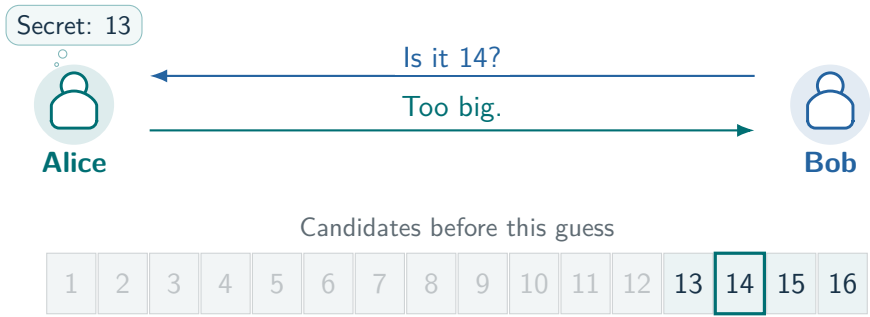
Guess the middle; keep the half indicated by Alice's reply.



Keep $13, \ldots, 16$.

Algorithm 2: Binary search

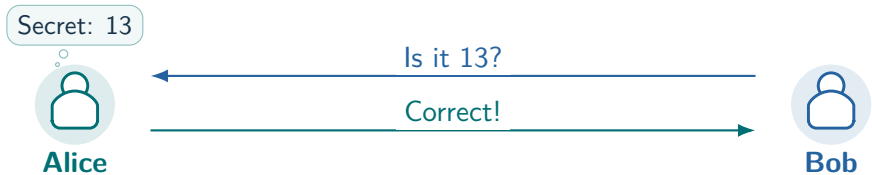
Guess the middle; keep the half indicated by Alice's reply.



Only 13 remains.

Algorithm 2: Binary search

Guess the middle; keep the half indicated by Alice's reply.



Four guesses instead of thirteen.

Why does binary search seem faster?

Algorithm 1: Linear search

Each failed guess rules out **one** new number.

$$n \longrightarrow n-1 \longrightarrow n-2 \longrightarrow \cdots$$

A large secret takes many guesses to reach.

Why does binary search seem faster?

Algorithm 1: Linear search

Each failed guess rules out **one** new number.

$$n \longrightarrow n-1 \longrightarrow n-2 \longrightarrow \cdots$$

A large secret takes many guesses to reach.

Algorithm 2: Binary search

Each failed guess rules out **at least half** the remaining numbers.

$$n \longrightarrow \leq n/2 \longrightarrow \leq n/4 \longrightarrow \cdots$$

The remaining range shrinks quickly.

How should we compare algorithms?

Same game, $n = 16$. Count guesses, including the correct guess.

Secret	Linear search		Binary search	
13	$1, 2, \ldots, 13$	(13 guesses)	$8, 12, 14, 13$	(4 guesses)

How should we compare algorithms?

Same game, $n = 16$. Count guesses, including the correct guess.

Secret	Linear search	Binary search
13	$1, 2, \ldots, 13$ (13 guesses)	$8, 12, 14, 13$ (4 guesses)
1	1 (1 guess)	$8, 4, 2, 1$ (4 guesses)

How should we compare algorithms?

Same game, $n = 16$. Count guesses, including the correct guess.

Secret	Linear search	Binary search
13	$1, 2, \ldots, 13$ (13 guesses)	$8, 12, 14, 13$ (4 guesses)
1	1 (1 guess)	$8, 4, 2, 1$ (4 guesses)

How should we compare different algorithms?

The number of guesses depends on the instance. How can we compare the algorithms as a whole?

1. **Problems and algorithms**

   Problem specification; instances; algorithms and correctness.

2. **Algorithm analysis**

   Basic operations; exact counts; worst-case running time.

3. **Asymptotic analysis**

   Growth rates; upper and lower bounds; tight bounds.

Running time: Count basic operations

Running time The total number of basic operations performed.

Running time: Count basic operations

Running time The total number of basic operations performed.

We charge **one unit of time** for each basic operation on machine words.

Operation	Example
Assignment	$L \leftarrow 1$
Comparison	$L \leq R$
Basic arithmetic	$+,\ -,\ \times,\ \div,\ \text{mod}$
Memory read or write	Read $A[i]$; store a value in $A[i]$

Running time: Count basic operations

Running time The total number of basic operations performed.

We charge **one unit of time** for each basic operation on machine words.

Operation	Example
Assignment	$L \leftarrow 1$
Comparison	$L \leq R$
Basic arithmetic	$+$, $-$, $\times$, $\div$, mod
Memory read or write	Read $A[i]$; store a value in $A[i]$

RAM model Random-access machine: accessing any memory location costs one unit of time.

Worst-case analysis

Different instances of the same size can take different amounts of time.

Worst-case analysis

Different instances of the same size can take different amounts of time.

Worst-case running time

The **largest** running time over all valid instances of size n.

$$T(n) = \max_{\text{instances } I \text{ of size } n} \text{time}(I).$$

Worst-case analysis

Different instances of the same size can take different amounts of time.

Worst-case running time

The **largest** running time over all valid instances of size n.

$$T(n) = \max_{\text{instances } I \text{ of size } n} \text{time}(I).$$

This lecture focuses on worst-case analysis.

Alice–Bob guessing: Running time

Alice's secret is $a \in \{1, \ldots, n\}$.

Our cost model for this game

Count only calls to AskAlice: each call costs one unit.

$$T(n, a) = \#\text{ calls to AskAlice}.$$

Alice–Bob guessing: Running time

Alice's secret is $a \in \{1, \ldots, n\}$.

Our cost model for this game

Count only calls to AskAlice: each call costs one unit.

$$T(n, a) = \#\text{ calls to AskAlice}.$$

For a fixed range $1, \ldots, n$, which secret a takes the most calls?

Guessing algorithms: Worst-case analysis

Secret $a \in \{1, \ldots, n\}$; count only calls to AskAlice.

Linear search

Guess $1, 2, 3, \ldots$ until correct.

Guessing algorithms: Worst-case analysis

Secret $a \in \{1, \ldots, n\}$; count only calls to AskAlice.

Linear search

Guess $1, 2, 3, \ldots$ until correct.

Which secret takes the most calls?

Guessing algorithms: Worst-case analysis

Secret $a \in \{1, \ldots, n\}$; count only calls to AskAlice.

Linear search

Guess $1, 2, 3, \ldots$ until correct.

Which secret takes the most calls?

$$T(n, a) = a.$$

Worst case: $a = n$

Bob makes n **calls**.

Guessing algorithms: Worst-case analysis

Secret $a \in \{1, \ldots, n\}$; count only calls to AskAlice.

Linear search

Guess $1, 2, 3, \ldots$ until correct.

Which secret takes the most calls?

$$T(n, a) = a.$$

Worst case: $a = n$

Bob makes n **calls**.

Binary search

Each wrong guess removes at least half:

$$n \longrightarrow \leq \frac{n}{2} \longrightarrow \leq \frac{n}{4} \longrightarrow \cdots$$

Guessing algorithms: Worst-case analysis

Secret $a \in \{1, \ldots, n\}$; count only calls to AskAlice.

Linear search

Guess $1, 2, 3, \ldots$ until correct.

Which secret takes the most calls?

$$T(n, a) = a.$$

Worst case: $a = n$

Bob makes n **calls**.

Binary search

Each wrong guess removes at least half:

$$n \longrightarrow \leq \frac{n}{2} \longrightarrow \leq \frac{n}{4} \longrightarrow \cdots$$

After $T - 1$ wrong guesses, at least one candidate remains:

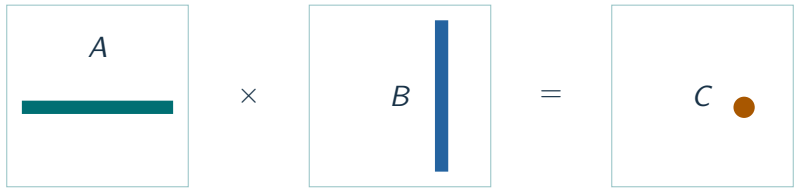
$$1 \leq \frac{n}{2^{T-1}}.$$

Worst-case bound

$$T \leq \log_2 n + 1 \textbf{ calls.}$$

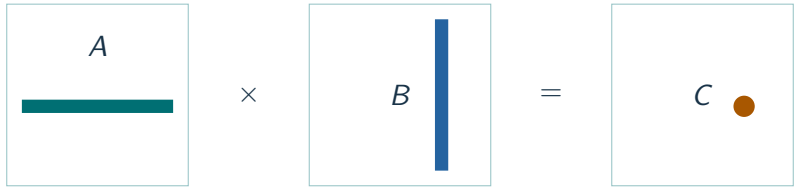
Case study: Matrix multiplication

For two $n \times n$ matrices, compute $C = AB$.



Case study: Matrix multiplication

For two $n \times n$ matrices, compute $C = AB$.



$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj} = a_{i1} b_{1j} + a_{i2} b_{2j} + \cdots + a_{in} b_{nj}.$$

Matrix multiplication: The standard algorithm

Count only scalar multiplications ($n \geq 1$).

Multiply(A, B)

for $i = 1, \ldots, n$
  for $j = 1, \ldots, n$
    $c_{ij} \leftarrow 0$
    for $k = 1, \ldots, n$
      $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$
return C

Matrix multiplication: The standard algorithm

Count only scalar multiplications ($n \geq 1$).

Multiply(A, B)

for $i = 1, \ldots, n$
  for $j = 1, \ldots, n$
    $c_{ij} \leftarrow 0$
    for $k = 1, \ldots, n$
      $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$
return C

One entry c_{ij}:

n multiplications.

Matrix multiplication: The standard algorithm

Count only scalar multiplications ($n \geq 1$).

Multiply(A, B)

for $i = 1, \ldots, n$
  for $j = 1, \ldots, n$
    $c_{ij} \leftarrow 0$
    for $k = 1, \ldots, n$
      $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$
return C

One entry c_{ij}:

n multiplications.

All n^2 entries:

$$T(n) = n^2 \times n = n^3.$$

What if we count every basic operation?

What about additions, assignments, and the loops themselves?

What if we count every basic operation?

What about additions, assignments, and the loops themselves?

Exact counting can be tedious.

What if we count every basic operation?

What about additions, assignments, and the loops themselves?

Exact counting can be tedious.

Asymptotic analysis

Focus on how running time **grows** as the input gets larger.

1. **Problems and algorithms**

   Problem specification; instances; algorithms and correctness.

2. **Algorithm analysis**

   Basic operations; exact counts; worst-case running time.

3. **Asymptotic analysis**

   Growth rates; upper and lower bounds; tight bounds.

Asymptotic analysis

Asymptotic analysis describes how a cost function grows as the input size $n \to \infty$.

Algorithm	Count	Main growth
Linear search, worst case	n	
Binary search, worst case	$\lfloor \log_2 n \rfloor + 1$	
Standard matrix product	n^3	

Asymptotic analysis

Asymptotic analysis describes how a cost function grows as the input size $n \to \infty$.

Algorithm	Count	Main growth
Linear search, worst case	n	linear
Binary search, worst case	$\lfloor \log_2 n \rfloor + 1$	logarithmic
Standard matrix product	n^3	cubic

Asymptotic analysis

Asymptotic analysis describes how a cost function grows as the input size $n \to \infty$.

Algorithm	Count	Main growth
Linear search, worst case	n	linear
Binary search, worst case	$\lfloor \log_2 n \rfloor + 1$	logarithmic
Standard matrix product	n^3	cubic

We want notation that ignores constant factors and lower-order terms.

Big-O: An upper bound for large inputs

MUST UNDERSTAND **Big-O**

For nonnegative functions f and g,

$$f(n) = O(g(n)) \quad \text{means} \quad f(n) \leq Cg(n) \text{ for all } n \geq n_0,$$

for some constants $C > 0$ and n_0.

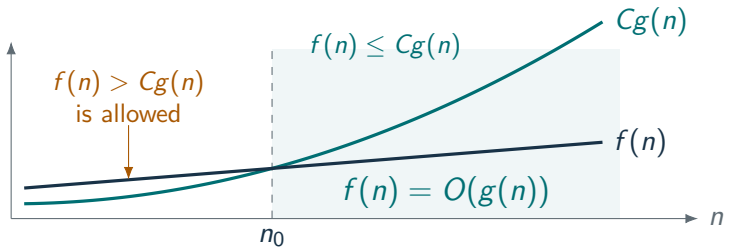
Big-O: An upper bound for large inputs

MUST UNDERSTAND **Big-O**

For nonnegative functions f and g,

$$f(n) = O(g(n)) \quad \text{means} \quad f(n) \leq Cg(n) \text{ for all } n \geq n_0,$$

for some constants $C > 0$ and n_0.



Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7		
$\log_2 n + 1$		
$3n + 10$		
$n \log_2 n + 4n$		
$5n^2 + 2n + 1$		
$2n^3 + 5n^2 + 1$		

Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$		
$3n + 10$		
$n \log_2 n + 4n$		
$5n^2 + 2n + 1$		
$2n^3 + 5n^2 + 1$		

Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$		
$n \log_2 n + 4n$		
$5n^2 + 2n + 1$		
$2n^3 + 5n^2 + 1$		

For any fixed base $b > 1$, $\log_b n = (\log_2 n)/\mathbf{(\log_2 b)}$,
only a **constant factor** away from $\log_2 n$. Big-O ignores constants, so we write $\log n$.

Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$	$O(n)$	**Linear**: grows with n
$n \log_2 n + 4n$		
$5n^2 + 2n + 1$		
$2n^3 + 5n^2 + 1$		

Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$	$O(n)$	**Linear**: grows with n
$n \log_2 n + 4n$	$O(n \log n)$	**Quasilinear**: linear with log factors
$5n^2 + 2n + 1$		
$2n^3 + 5n^2 + 1$		

Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$	$O(n)$	**Linear**: grows with n
$n \log_2 n + 4n$	$O(n \log n)$	**Quasilinear**: linear with log factors
$5n^2 + 2n + 1$	$O(n^2)$	**Quadratic**: grows with n^2
$2n^3 + 5n^2 + 1$		

Big-O: Examples of functions

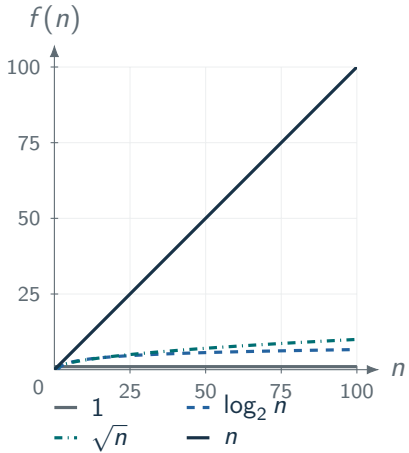
Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$	$O(n)$	**Linear**: grows with n
$n \log_2 n + 4n$	$O(n \log n)$	**Quasilinear**: linear with log factors
$5n^2 + 2n + 1$	$O(n^2)$	**Quadratic**: grows with n^2
$2n^3 + 5n^2 + 1$	$O(n^3)$	**Cubic**: grows with n^3

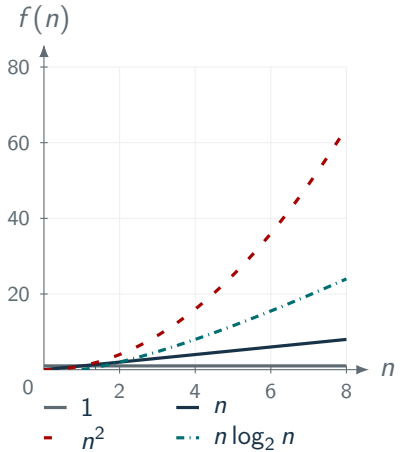
Big-O: Examples of functions

Function $f(n)$	A Big-O bound	Growth type / note
7	$O(1)$	**Constant**: independent of n
$\log_2 n + 1$	$O(\log n)$	**Logarithmic**: grows like $\log n$
$3n + 10$	$O(n)$	**Linear**: grows with n
$n \log_2 n + 4n$	$O(n \log n)$	**Quasilinear**: linear with log factors
$5n^2 + 2n + 1$	$O(n^2)$	**Quadratic**: grows with n^2
$2n^3 + 5n^2 + 1$	$O(n^3)$	**Cubic**: grows with n^3

For example, n^2 is also $O(n^3)$ and $O(n^4)$: an upper bound need not be tight.

Comparing growth rates





Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$		
$n! + 2^n$		
$n^2(2 + (-1)^n)$		
$\sqrt{n^2 + n}$		
2^{n+10}		
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$		
$n^2(2 + (-1)^n)$		
$\sqrt{n^2 + n}$		
2^{n+10}		
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$	$O(n!)$	$2^n \leq n!$ for $n \geq 4$
$n^2(2 + (-1)^n)$		
$\sqrt{n^2 + n}$		
2^{n+10}		
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$	$O(n!)$	$2^n \leq n!$ for $n \geq 4$
$n^2(2 + (-1)^n)$	$O(n^2)$	Always n^2 or $3n^2$
$\sqrt{n^2 + n}$		
2^{n+10}		
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$	$O(n!)$	$2^n \leq n!$ for $n \geq 4$
$n^2(2 + (-1)^n)$	$O(n^2)$	Always n^2 or $3n^2$
$\sqrt{n^2 + n}$	$O(n)$	$\sqrt{n^2 + n} \leq \sqrt{2}\, n$ for $n \geq 1$
2^{n+10}		
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$	$O(n!)$	$2^n \leq n!$ for $n \geq 4$
$n^2(2 + (-1)^n)$	$O(n^2)$	Always n^2 or $3n^2$
$\sqrt{n^2 + n}$	$O(n)$	$\sqrt{n^2 + n} \leq \sqrt{2}\, n$ for $n \geq 1$
2^{n+10}	$O(2^n)$	$2^{n+10} = 1024 \cdot 2^n$
2^{2n}		

Big-O: Less obvious examples

Function $f(n)$	A Big-O bound	Why?
$n!$	$O(n^n)$	$1 \times 2 \times \cdots \times n \leq n^n$
$n! + 2^n$	$O(n!)$	$2^n \leq n!$ for $n \geq 4$
$n^2(2 + (-1)^n)$	$O(n^2)$	Always n^2 or $3n^2$
$\sqrt{n^2 + n}$	$O(n)$	$\sqrt{n^2 + n} \leq \sqrt{2}\, n$ for $n \geq 1$
2^{n+10}	$O(2^n)$	$2^{n+10} = 1024 \cdot 2^n$
2^{2n}	$O(4^n)$, but not $O(2^n)$	$2^{2n}/2^n = 2^n \to \infty$

Matrix multiplication: Why use asymptotic analysis?

Multiply(A, B)

for $i = 1, \ldots, n$
  for $j = 1, \ldots, n$
    $c_{ij} \leftarrow 0$
    for $k = 1, \ldots, n$
      $c_{ij} \leftarrow c_{ij} + a_{ik} b_{kj}$
return C

Matrix multiplication: Why use asymptotic analysis?

Multiply(A, B)

```
for i = 1, ..., n
    for j = 1, ..., n
        c_ij ← 0
        for k = 1, ..., n
            c_ij ← c_ij + a_ik b_kj
return C
```

Detailed counting

Operation	Count
Multiplications	n^3
Additions to c_{ij}	n^3
Assignments to c_{ij}	$n^3 + n^2$

Plus loop and memory operations.

Matrix multiplication: Why use asymptotic analysis?

Multiply(A, B)

```
for i = 1, ..., n
    for j = 1, ..., n
        c_ij ← 0
        for k = 1, ..., n
            c_ij ← c_ij + a_ik b_kj
return C
```

Detailed counting

Operation	Count
Multiplications	n^3
Additions to c_{ij}	n^3
Assignments to c_{ij}	$n^3 + n^2$

Plus loop and memory operations.

Running time

$$T(n) = O(n^3)$$

Case study: Finding duplication

Problem: Find a repeated value in $A[1..n]$, or report no match.

Instance: $A = [1, 2, 5, 6, 2, 10]$. Return 2.

Case study: Finding duplication

Problem: Find a repeated value in $A[1..n]$, or report no match.

Instance: $A = [1, 2, 5, 6, 2, 10]$. Return 2.

Algorithm 1: Try every pair

for $i = 1, \ldots, n-1$
  for $j = i+1, \ldots, n$
    if $A[i] = A[j]$:
      return $A[i]$
return No-Match

Case study: Finding duplication

Problem: Find a repeated value in $A[1..n]$, or report no match.

Instance: $A = [1, 2, 5, 6, 2, 10]$. Return 2.

Algorithm 1: Try every pair

```
for i = 1, ..., n − 1
   for j = i + 1, ..., n
      if A[i] = A[j]:
         return A[i]
return No-Match
```

Worst case: all values are different.

Compare the first entry with $n-1$ others, the next with $n-2$, and so on.

$$W(n) = (n-1) + (n-2) + \cdots + 1$$
$$= \frac{n(n-1)}{2} \leq n^2.$$

Time: $O(n^2)$.

Case study: Finding duplication

Assumption: Every value is an integer from 1 to n.

Pseudocode

```
SeenBefore[1..n] ← "No"
for each x in A
   if SeenBefore[x] = "Yes":
      return x
   SeenBefore[x] ← "Yes"
return No-Match
```

Case study: Finding duplication

Assumption: Every value is an integer from 1 to n.

Pseudocode

```
SeenBefore[1..n] ← "No"
for each x in A
   if SeenBefore[x] = "Yes":
      return x
   SeenBefore[x] ← "Yes"
return No-Match
```

"Yes" means the value appeared earlier.

Initialize n entries: $O(n)$.

Scan n entries, constant work each: $O(n)$.

Time: $O(n) + O(n) = O(n)$.

Extra space: $O(n)$.

Case study: Finding duplication

Assumption: Every value is an integer from 1 to n.

Pseudocode

$SeenBefore[1..n] \leftarrow$ "No"
for each x **in** A
  if $SeenBefore[x] =$ "Yes":
    return x
  $SeenBefore[x] \leftarrow$ "Yes"
return NO-MATCH

"Yes" means the value appeared earlier.

Initialize n entries: $O(n)$.

Scan n entries, constant work each: $O(n)$.

Time: $O(n) + O(n) = O(n)$.

Extra space: $O(n)$.

Exercise

What if the values are not bounded? Can you still find a duplicate in $O(n)$ time?

Upper bound, lower bound, tight bound

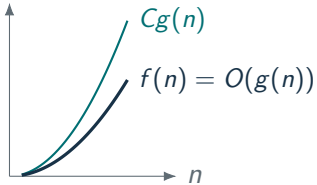
For nonnegative f, g, choose positive constants and a threshold n_0. The inequality must hold for **every** $n \geq n_0$.

Upper bound

$$f(n) = \boldsymbol{O}(g(n))$$

Read as "big O"

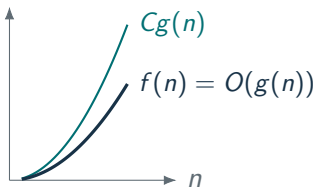
$$f(n) \leq Cg(n)$$

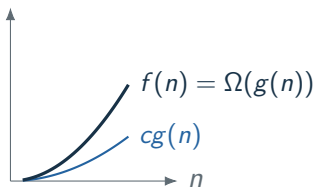


Upper bound, lower bound, tight bound

For nonnegative f, g, choose positive constants and a threshold n_0. The inequality must hold for **every** $n \geq n_0$.

Upper bound	Lower bound
$f(n) = \boldsymbol{O}(g(n))$	$f(n) = \boldsymbol{\Omega}(g(n))$
Read as "big O"	Read as "Omega"
$f(n) \leq Cg(n)$	$cg(n) \leq f(n)$

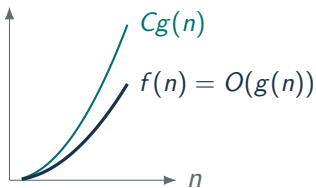


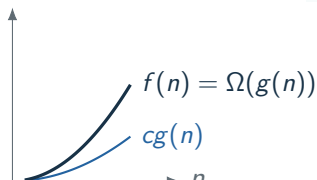


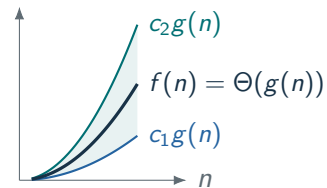
Upper bound, lower bound, tight bound

For nonnegative f, g, choose positive constants and a threshold n_0. The inequality must hold for **every** $n \geq n_0$.

Upper bound	Lower bound	Tight bound
$f(n) = \boldsymbol{O}(g(n))$	$f(n) = \boldsymbol{\Omega}(g(n))$	$f(n) = \boldsymbol{\Theta}(g(n))$
Read as "big O"	Read as "Omega"	Read as "Theta"
$f(n) \leq Cg(n)$	$cg(n) \leq f(n)$	$c_1 g(n) \leq f(n) \leq c_2 g(n)$







Strict bounds

For $f(n) \geq 0$ and $g(n) > 0$:

Strict upper bound

$$f(n) = \boldsymbol{o}(g(n)) \quad \text{“little o”}$$
$$\iff \quad \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Example

$$100n^2 = \boldsymbol{o}(n^3), \qquad \frac{100n^2}{n^3} = \frac{100}{n} \to 0.$$

Strict bounds

For $f(n) \geq 0$ and $g(n) > 0$:

Strict upper bound

$$f(n) = \boldsymbol{o}(g(n)) \quad \text{“little o”}$$
$$\iff \quad \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Example

$$100n^2 = \boldsymbol{o}(n^3), \qquad \frac{100n^2}{n^3} = \frac{100}{n} \to 0.$$

Strict lower bound

$$f(n) = \boldsymbol{\omega}(g(n)) \quad \text{“little omega”}$$
$$\iff \quad \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Example

$$n^3 = \boldsymbol{\omega}(100n^2), \qquad \frac{n^3}{100n^2} = \frac{n}{100} \to \infty.$$

Proving a tight bound

Match the upper and lower bounds

$$f(n) = O(g(n)) \quad \text{and} \quad f(n) = \Omega(g(n))$$
$$\implies \quad f(n) = \Theta(g(n)).$$

If the upper bound matches the lower bound, we get a **tight bound**.

Geometric series

For an integer $k \geq 0$ and a real number $r \neq 1$,

Geometric-series sum

$$S = 1 + r + r^2 + \cdots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

Geometric series

For an integer $k \geq 0$ and a real number $r \neq 1$,

Geometric-series sum

$$S = 1 + r + r^2 + \cdots + r^k = \frac{r^{k+1} - 1}{r - 1}.$$

Proof: Multiply by r, then subtract:

$$rS = r + r^2 + \cdots + r^k + r^{k+1},$$
$$S = 1 + r + r^2 + \cdots + r^k,$$
$$(r-1)S = r^{k+1} - 1.$$

Geometric series: Counting work across stages

An algorithm does $1, 2, 4, \ldots, n$ operations in successive stages, where $n = 2^k$.



Geometric series: Counting work across stages

An algorithm does $1, 2, 4, \ldots, n$ operations in successive stages, where $n = 2^k$.



$$T(n) = 1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 = 2n - 1.$$

Geometric series: Counting work across stages

An algorithm does $1, 2, 4, \ldots, n$ operations in successive stages, where $n = 2^k$.



$$T(n) = 1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1 = 2n - 1.$$

Total work

$n \leq T(n) < 2n$, so $\boldsymbol{T(n) = \Theta(n)}$.

Case study: Harmonic sum

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Case study: Harmonic sum

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

Case study: Harmonic sum

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

For $n = 2^k$, group the terms into $k = \log_2 n$ blocks:

$$H_n = 1 + \underbrace{\frac{1}{2}}_{1 \text{ term}} + \underbrace{\frac{1}{3} + \frac{1}{4}}_{2 \text{ terms}} + \underbrace{\frac{1}{5} + \cdots + \frac{1}{8}}_{4 \text{ terms}} + \cdots .$$

Case study: Harmonic sum

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

For $n = 2^k$, group the terms into $k = \log_2 n$ blocks:

$$H_n = 1 + \underbrace{\frac{1}{2}}_{1 \text{ term}} + \underbrace{\frac{1}{3} + \frac{1}{4}}_{2 \text{ terms}} + \underbrace{\frac{1}{5} + \cdots + \frac{1}{8}}_{4 \text{ terms}} + \cdots.$$

Upper bound

Each block sums to at most 1:

$$\frac{1}{5} + \cdots + \frac{1}{8} \leq 4 \cdot \frac{1}{4} = 1.$$

$H_n \leq 1 + k$, so $H_n = O(\log n)$.

Case study: Harmonic sum

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

For $n = 2^k$, group the terms into $k = \log_2 n$ blocks:

$$H_n = 1 + \underbrace{\frac{1}{2}}_{1 \text{ term}} + \underbrace{\frac{1}{3} + \frac{1}{4}}_{2 \text{ terms}} + \underbrace{\frac{1}{5} + \cdots + \frac{1}{8}}_{4 \text{ terms}} + \cdots.$$

Upper bound

Each block sums to at most 1:

$$\frac{1}{5} + \cdots + \frac{1}{8} \leq 4 \cdot \frac{1}{4} = 1.$$

$H_n \leq 1 + k$, so $H_n = O(\log n)$.

Lower bound

Each block sums to at least $1/2$:

$$\frac{1}{5} + \cdots + \frac{1}{8} \geq 4 \cdot \frac{1}{8} = \frac{1}{2}.$$

$H_n \geq 1 + k/2$, so $H_n = \Omega(\log n)$.

Case study: Harmonic sum — general n

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

Upper bound: $n = 2^k$

$$H_n = O(\log n)$$

Lower bound: $n = 2^k$

$$H_n = \Omega(\log n)$$

Thus $H_n = \Theta(\log n)$ for powers of 2.

Case study: Harmonic sum — general n

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

Upper bound: $n = 2^k$

$$H_n = O(\log n)$$

Lower bound: $n = 2^k$

$$H_n = \Omega(\log n)$$

Thus $H_n = \Theta(\log n)$ for powers of 2.

For any $n \geq 2$, choose a power of two M with $M \leq n < 2M$.

$$\underbrace{H_M}_{\Theta(\log M)} \leq H_n \leq \underbrace{H_{2M}}_{\Theta(\log(2M))}.$$

Case study: Harmonic sum — general n

Definition: The harmonic sum is $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$.

Theorem: $\boldsymbol{H_n = \Theta(\log n)}$.

Upper bound: $n = 2^k$

$$H_n = O(\log n)$$

Lower bound: $n = 2^k$

$$H_n = \Omega(\log n)$$

Thus $H_n = \Theta(\log n)$ for powers of 2.

For any $n \geq 2$, choose a power of two M with $M \leq n < 2M$.

$$\underbrace{H_M}_{\Theta(\log M)} \leq H_n \leq \underbrace{H_{2M}}_{\Theta(\log(2M))}.$$

Since $\log_2 M \leq \log_2 n < \log_2 M + 1$, both endpoints have order $\log n$.

$$H_n = \Theta(\log n).$$

Case study: A sum of logarithms

Claim:

$$f(n) = \log_2 1 + \log_2 2 + \cdots + \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Case study: A sum of logarithms

Claim:

$$f(n) = \log_2 1 + \log_2 2 + \cdots + \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Upper bound

$$f(n) \leq \underbrace{\log_2 n + \cdots + \log_2 n}_{n \text{ terms}}$$
$$= n \log_2 n.$$
$$f(n) = O(n \log n)$$

Case study: A sum of logarithms

Claim:

$$f(n) = \log_2 1 + \log_2 2 + \cdots + \log_2 n = \boldsymbol{\Theta(n \log n)}.$$

Upper bound

$$f(n) \leq \underbrace{\log_2 n + \cdots + \log_2 n}_{n \text{ terms}}$$
$$= n \log_2 n.$$
$$f(n) = O(n \log n)$$

Lower bound

$$f(n) \geq \underbrace{\log_2 \lceil n/2 \rceil + \cdots + \log_2 n}_{\text{latter half: at least } n/2 \text{ terms}}$$
$$\geq \frac{n}{2} \log_2 \frac{n}{2}.$$
$$f(n) = \Omega(n \log n)$$

Summary

1. **Problems and algorithms.**
   A problem specifies inputs and outputs; an instance is one input.
   An algorithm must solve every permitted instance.

2. **Running time.**
   Choose a size, an operation, and a case; derive the count.

3. **Asymptotic analysis.**
   Use O for upper bounds, Ω for lower bounds, and Θ for tight bounds.

Review exercises

1. Trace binary guessing with $n = 16$ and secrets 2 and 15.
2. Give constants proving $4n^2 + 3n + 8 = \Theta(n^2)$.
3. Count the body executions in

   for $i = 1, \ldots, n$: **for** $j = 1, \ldots, n - i$: body.

Additional examples and perspectives

Fast exponentiation: Repeated squaring

Compute $a^b \bmod m$, where $b \geq 1$ and $m \geq 2$. Let $L = \lfloor \log_2 b \rfloor + 1$.

1. Binary decomposition

for $i = 0, \ldots, L-1$
  $b_i \leftarrow b \bmod 2; \quad b \leftarrow b \div 2$

2. Accumulate

$r \leftarrow 1; \quad sqr \leftarrow a \bmod m$
for $i = 0, \ldots, L-1$
  if $b_i = 1$: $r \leftarrow r \times sqr \bmod m$
  $sqr \leftarrow sqr^2 \bmod m$
return r

Fast exponentiation: Repeated squaring

Compute $a^b \bmod m$, where $b \geq 1$ and $m \geq 2$. Let $L = \lfloor \log_2 b \rfloor + 1$.

1. Binary decomposition

for $i = 0, \ldots, L-1$
  $b_i \leftarrow b \bmod 2; \quad b \leftarrow b \div 2$

2. Accumulate

$r \leftarrow 1; \quad sqr \leftarrow a \bmod m$
for $i = 0, \ldots, L-1$
  if $b_i = 1$: $r \leftarrow r \times sqr \bmod m$
  $sqr \leftarrow sqr^2 \bmod m$
return r

1. Extract the bits

L iterations; constant work each.

2. Accumulate the answer

L iterations; at most two modular multiplications each.

Fast exponentiation: Repeated squaring

Compute $a^b \bmod m$, where $b \geq 1$ and $m \geq 2$. Let $L = \lfloor \log_2 b \rfloor + 1$.

1. Binary decomposition

for $i = 0, \ldots, L-1$
  $b_i \leftarrow b \bmod 2; \quad b \leftarrow b \div 2$

2. Accumulate

$r \leftarrow 1; \quad sqr \leftarrow a \bmod m$
for $i = 0, \ldots, L-1$
  if $b_i = 1$: $r \leftarrow r \times sqr \bmod m$
  $sqr \leftarrow sqr^2 \bmod m$
return r

1. Extract the bits

L iterations; constant work each.

2. Accumulate the answer

L iterations; at most two modular multiplications each.

Running time

$$O(L) = O(\log b)$$

Count each arithmetic operation as one unit.

Matrix multiplication: Beyond cubic time

The standard algorithm uses $\Theta(n^3)$ arithmetic operations.

OpenAI, October 2026 [1]

Proves the improved exponent bound using a nonconstructive argument.

Matrix multiplication: Beyond cubic time

The standard algorithm uses $\Theta(n^3)$ arithmetic operations.

OpenAI, October 2026 [1]

Proves the improved exponent bound using a nonconstructive argument.

Rafael Pass, October 2026 [2]

Gives an explicit recursive construction matching the bound.

Matrix multiplication: Beyond cubic time

The standard algorithm uses $\Theta(n^3)$ arithmetic operations.

OpenAI, October 2026 [1]

Proves the improved exponent bound using a nonconstructive argument.

Rafael Pass, October 2026 [2]

Gives an explicit recursive construction matching the bound.

For every fixed $\varepsilon > 0$, the construction multiplies integer matrices in

$$O(n^{2.25+\varepsilon}) \text{ arithmetic operations.}$$

Input length: Digits versus numerical value

An integer $N \geq 1$ has $m = \lfloor \log_2 N \rfloor + 1$ binary digits.



Input length: Digits versus numerical value

An integer $N \geq 1$ has $m = \lfloor \log_2 N \rfloor + 1$ binary digits.



A loop with N iterations can therefore take exponentially many steps in m.

Input length: Digits versus numerical value

An integer $N \geq 1$ has $m = \lfloor \log_2 N \rfloor + 1$ binary digits.



A loop with N iterations can therefore take exponentially many steps in m.

Polynomial time

Worst-case time $O(m^k)$ for some fixed constant k, where m is the encoded input length.

Linear search: Steps and justification

Input: Array $A[1..n]$, target x. **Output:** A matching index or NOT-FOUND.

LinearSearch(A, x)

```
for i = 1, ..., n
   if A[i] = x
      return i
return NOT-FOUND
```

Linear search: Steps and justification

Input: Array $A[1..n]$, target x. **Output:** A matching index or NOT-FOUND.

LinearSearch(A, x)

```
for i = 1, ..., n
   if A[i] = x
      return i
return NOT-FOUND
```

Correctness

Before checking index i, every earlier entry has been ruled out.

Linear search: Steps and justification

Input: Array $A[1..n]$, target x. **Output:** A matching index or NOT-FOUND.

LinearSearch(A, x)

```
for i = 1, ..., n
  if A[i] = x
    return i
return NOT-FOUND
```

Correctness

Before checking index i, every earlier entry has been ruled out.

A match gives a valid index; reaching the end rules out every entry.

Linear search: Steps and justification

Input: Array $A[1..n]$, target x. **Output:** A matching index or NOT-FOUND.

LinearSearch(A, x)

```
for i = 1, ..., n
  if A[i] = x
    return i
return NOT-FOUND
```

Correctness

Before checking index i, every earlier entry has been ruled out.

A match gives a valid index; reaching the end rules out every entry.

Termination

There are at most n checks.

Binary search: Use sorted order

Precondition: A is sorted in nondecreasing order. Find $x = 42$.



$A[5] = 19 < 42$. Every entry to its left is also too small.

Keep indices 6 through 10.

Binary search: Use sorted order

Precondition: A is sorted in nondecreasing order. Find $x = 42$.



$A[8] = 54 > 42$. Every entry to its right is also too large.

Keep indices 6 through 7.

Binary search: Use sorted order

Precondition: A is sorted in nondecreasing order. Find $x = 42$.



$A[6] = 20 < 42$. Keep index 7.

Binary search: Use sorted order

Precondition: A is sorted in nondecreasing order. Find $x = 42$.



$A[7] = 42$. Return 7.

Binary search: The full algorithm

Input: sorted array $A[1..n]$, target x.

BinarySearch(A, x)

$L \leftarrow 1$; $R \leftarrow n$
while $L \leq R$
    $M \leftarrow \lfloor (L + R)/2 \rfloor$
    if $A[M] = x$: **return** M
    if $A[M] < x$: $L \leftarrow M + 1$
    else: $R \leftarrow M - 1$
return NOT-FOUND

Binary search: Why it works

What remains true after each step

If x occurs in the array, an occurrence lies in $A[L..R]$.

Binary search: Why it works

What remains true after each step

If x occurs in the array, an occurrence lies in $A[L..R]$.

1. Initially, $[L, R] = [1, n]$ contains every entry.
2. Sorted order makes every discarded entry impossible.
3. If $L > R$, no possible position remains.

Binary search: Why it works

What remains true after each step

If x occurs in the array, an occurrence lies in $A[L..R]$.

1. Initially, $[L, R] = [1, n]$ contains every entry.
2. Sorted order makes every discarded entry impossible.
3. If $L > R$, no possible position remains.

Every failed probe removes the midpoint and shortens the interval, so the algorithm stops.

Insertion sort: Maintain a sorted prefix

Sort $[7, 3, 5, 2]$ into nondecreasing order.

Step	Sorted prefix	Unprocessed
Start	$[7]$	$[3, 5, 2]$

Insertion sort: Maintain a sorted prefix

Sort $[7, 3, 5, 2]$ into nondecreasing order.

Step	Sorted prefix	Unprocessed
Start	$[7]$	$[3, 5, 2]$
Insert 3	$[3, 7]$	$[5, 2]$

Insertion sort: Maintain a sorted prefix

Sort $[7, 3, 5, 2]$ into nondecreasing order.

Step	Sorted prefix	Unprocessed
Start	$[7]$	$[3, 5, 2]$
Insert 3	$[3, 7]$	$[5, 2]$
Insert 5	$[3, 5, 7]$	$[2]$

Insertion sort: Maintain a sorted prefix

Sort $[7, 3, 5, 2]$ into nondecreasing order.

Step	Sorted prefix	Unprocessed
Start	$[7]$	$[3, 5, 2]$
Insert 3	$[3, 7]$	$[5, 2]$
Insert 5	$[3, 5, 7]$	$[2]$
Insert 2	$[2, 3, 5, 7]$	$[\,]$

Reverse order requires $1 + 2 + \cdots + (n-1) = \Theta(n^2)$ comparisons.

Bubble sort: Move the largest item to the end

One left-to-right pass through $[7, 3, 5, 2]$:

Compare	Array after comparing and swapping if needed
$7, 3$	$[3, 7, 5, 2]$

Bubble sort: Move the largest item to the end

One left-to-right pass through $[7, 3, 5, 2]$:

Compare	Array after comparing and swapping if needed
$7, 3$	$[3, 7, 5, 2]$
$7, 5$	$[3, 5, 7, 2]$

Bubble sort: Move the largest item to the end

One left-to-right pass through $[7, 3, 5, 2]$:

Compare	Array after comparing and swapping if needed
$7, 3$	$[3, 7, 5, 2]$
$7, 5$	$[3, 5, 7, 2]$
$7, 2$	$[3, 5, 2, 7]$

Bubble sort: Move the largest item to the end

One left-to-right pass through $[7, 3, 5, 2]$:

Compare	Array after comparing and swapping if needed
$7, 3$	$[3, 7, 5, 2]$
$7, 5$	$[3, 5, 7, 2]$
$7, 2$	$[3, 5, 2, 7]$

Repeat on the remaining prefix. The fixed-pass version uses

$$(n-1) + (n-2) + \cdots + 1 = \Theta(n^2)$$

comparisons on every input.

String matching: Try each alignment

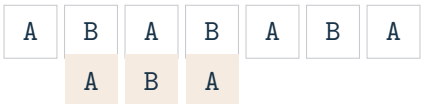
Find all occurrences of pattern `ABA` in text `ABABABA`.



Start at 1: match.

String matching: Try each alignment

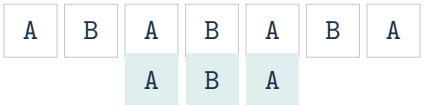
Find all occurrences of pattern `ABA` in text `ABABABA`.



Start at 2: first character mismatches.

String matching: Try each alignment

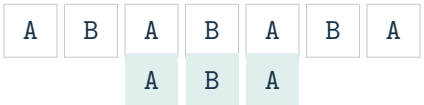
Find all occurrences of pattern `ABA` in text `ABABABA`.



Start at 3: match. Continue to starts 4 and 5.

String matching: Try each alignment

Find all occurrences of pattern `ABA` in text `ABABABA`.



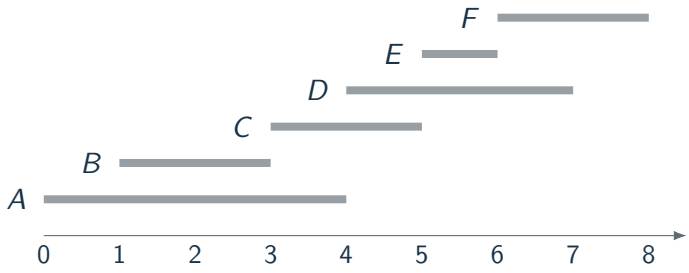
Start at 3: match. Continue to starts 4 and 5.

For $1 \leq m \leq n$: $n - m + 1$ alignments, at most m comparisons each.

$$W(n, m) \leq (n - m + 1)m = O(nm).$$

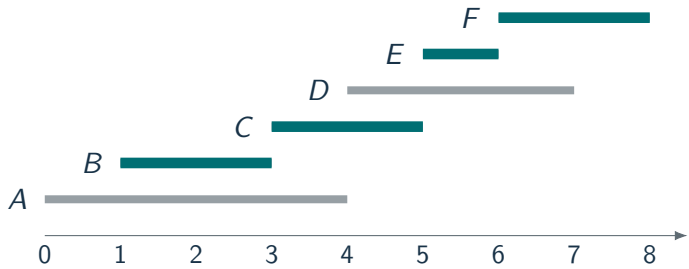
Greedy scheduling: Choose the most talks

Choose as many nonoverlapping talks as possible. Endpoints may meet.



Greedy scheduling: Choose the most talks

Choose as many nonoverlapping talks as possible. Endpoints may meet.



Choose the compatible talk that **finishes earliest**: B, C, E, F.

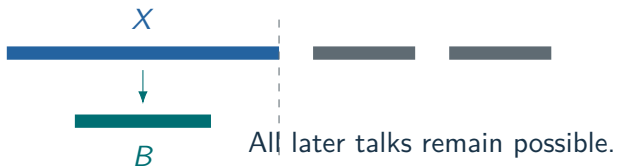
Why earliest finish works

Let B be the earliest-finishing talk. An optimal schedule starts with some talk X.



Why earliest finish works

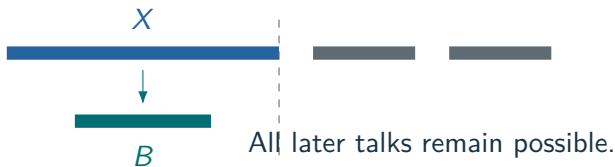
Let B be the earliest-finishing talk. An optimal schedule starts with some talk X.



Replace X by B: B finishes no later than X, so no later talk is lost.

Why earliest finish works

Let B be the earliest-finishing talk. An optimal schedule starts with some talk X.



Replace X by B: B finishes no later than X, so no later talk is lost. Repeat the argument on the talks that start after B finishes.

A greedy choice can fail

Make value 6 using coin values $1, 3, 4$, with as few coins as possible.

Largest affordable coin first

$$4 + 1 + 1 = 6$$

Three coins.

A greedy choice can fail

Make value 6 using coin values $1, 3, 4$, with as few coins as possible.

Largest affordable coin first

$$4 + 1 + 1 = 6$$

Three coins.

A better solution

$$3 + 3 = 6$$

Two coins.

Useful sums in algorithm analysis

$$\sum_{i=1}^{n} 1 = n = \Theta(n),$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2),$$

$$\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n),$$

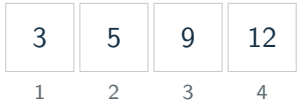
$$\sum_{i=0}^{n-1} c^i = \frac{c^n - 1}{c - 1} = \Theta(c^n) \quad (c > 1 \text{ fixed}).$$

Algorithm design patterns

Pattern	Idea	Example
Brute force	Try all candidates	Naive string matching
Greedy	Commit to a local choice	Earliest-finish scheduling
Divide and conquer	Split, solve, combine	Sort two halves, then merge
Dynamic programming	Store repeated subproblems	Store earlier Fibonacci values

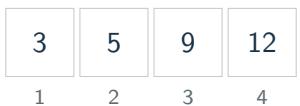
Finding an answer versus checking one

Subset sum: Do some of $3, 5, 9, 12$ sum to 17?



Finding an answer versus checking one

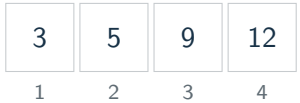
Subset sum: Do some of $3, 5, 9, 12$ sum to 17?



A proposed certificate is the subset of indices $\{2, 4\}$.

Finding an answer versus checking one

Subset sum: Do some of $3, 5, 9, 12$ sum to 17?



A proposed certificate is the subset of indices $\{2, 4\}$.

$$A[2] + A[4] = 5 + 12 = 17.$$

Check that the indices are valid and distinct, then add their values.

P and NP

Decision problems

P: solvable in polynomial time.

NP: yes-instances have polynomial-length certificates that can be verified in polynomial time.

P and NP

Decision problems

P: solvable in polynomial time.

NP: yes-instances have polynomial-length certificates that can be verified in polynomial time.

$\mathrm{P} \subseteq \mathrm{NP}$. Whether $\mathrm{P} = \mathrm{NP}$ is open.

Polynomial-time reductions

$X \leq_p Y$: transform an instance x to $R(x)$ in polynomial time, preserving the yes/no answer.

$$x \in X \quad \Longleftrightarrow \quad R(x) \in Y.$$

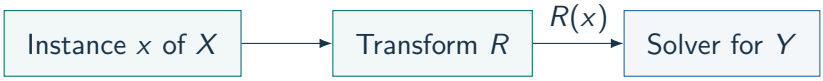
Polynomial-time reductions

$X \leq_p Y$: transform an instance x to $R(x)$ in polynomial time, preserving the yes/no answer.

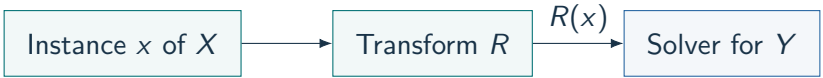
$$x \in X \quad \iff \quad R(x) \in Y.$$

A polynomial-time solver for Y would then solve X in polynomial time.

NP-completeness: Why the direction matters

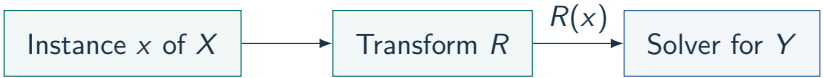


NP-completeness: Why the direction matters



An **NP-complete** problem is in NP, and every problem in NP reduces to it in polynomial time.

NP-completeness: Why the direction matters



An **NP-complete** problem is in NP, and every problem in NP reduces to it in polynomial time.

A polynomial-time solver for one NP-complete problem would imply $\mathrm{P} = \mathrm{NP}$.

Some problems have no general algorithm

The halting problem

Given a program P and an input x, will P eventually stop on x?

Some problems have no general algorithm

The halting problem

Given a program P and an input x, will P eventually stop on x?

No algorithm answers correctly for every program-input pair.

Some problems have no general algorithm

The halting problem

Given a program P and an input x, will P eventually stop on x?

No algorithm answers correctly for every program-input pair.

An exponential-time algorithm

Terminates, but may require enormous time.

An undecidable problem

Has no algorithm that always terminates with the correct answer.

References

[1] OpenAI.

An upper bound of 9/4 for the matrix multiplication exponent, October 2026.

October 2. Official preprint.

[2] Rafael Pass.

Matrix multiplication from polynomial packings: A constructive and elementary proof of $\omega \leq 9/4$.

arXiv:2610.28627v2, October 2026.

Version 2, October 30.